実行ログに基づく修正を備えた サイバー演習環境と実行コードの自動生成パイプライン

城 明日香[1]　毛利 公美[2]　瀧田 愼[1]　葛野 弘樹[1]　古本 啓祐[3]　高橋 健志[3]　白石 善明[1,a)]

概要：サイバー攻撃の増加や複雑化に伴いハンズオン形式の演習の需要が高まる一方，人手による構築のコストは大きい．演習環境と実行コードは相互に依存するため整合させる必要があるが，関連研究では両者の整合した生成が十分に検討されていない．そこで，プロファイル群を入力として，LLM により Heat Orchestration Template（HOT）と実行コードを生成するパイプラインを提案する．このパイプラインは，OpenStack 上での展開・実行結果に基づいて HOT または実行コードを修正し，外部通信を遮断して隔離の成立を複数の方法で検証する．HOT 生成・実行コード生成・原因分析の各 STEP 間の制御は，人手を介さず Python オーケストレータが担う．初期侵入 3 種・Linux 権限昇格 2 種を組み合わせた 6 シナリオに適用し，修正ループを有効にした 60 試行のうち，再生成なしの成功は 36 試行，最終成功は 59 試行であった．

キーワード：サイバーレンジ，Infrastructure as Code，大規模言語モデル，実行ログ，OpenStack

Automatic Generation and Repair of Cyber Range Environments and Execution Scripts Based on Execution Logs

Asuka Shiro[1]　Masami Mohri[2]　Makoto Takita[1]　Hiroki Kuzuno[1]　Keisuke Furumoto[3]
Takeshi Takahashi[3]　Yoshiaki Shiraishi[1,a)]

Abstract: As cyberattacks increase in frequency and sophistication, demand for hands-on cybersecurity exercises is growing, yet constructing them manually is costly. The exercise environment and execution code are interdependent and must be kept consistent, yet existing work has not sufficiently addressed generating both consistently. We propose a pipeline that takes a set of profiles as input and uses an LLM to generate a Heat Orchestration Template (HOT) and execution code. The pipeline repairs the HOT or execution code based on deployment and execution results on OpenStack, and also verifies that isolation holds by cutting off external communication and checking it through multiple methods. A Python orchestrator, without human intervention, controls the transitions between the steps of HOT generation, execution-code generation, and failure diagnosis. We applied the pipeline to six scenarios combining three initial-access vectors and two Linux privilege-escalation techniques: of the 60 trials, all run with the repair loop enabled, 36 succeeded without regeneration and 59 succeeded in total.

Keywords: cyber range, infrastructure as code, large language models, execution logs, OpenStack

[1] 神戸大学
Kobe University
[2] 近畿大学
Kindai University
[3] 国立研究開発法人情報通信研究機構
National Institute of Information and Communications Technology
[a)] zenmei@port.kobe-u.ac.jp

1. はじめに

サイバー攻撃の増加と高度化に対応するため，セキュリティ人材を育成するサイバー演習が実施されている．なかでもハンズオン形式のサイバー演習は，受講者の自己効力感を高めることが報告されている [2]．一方で，人手による

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

演習環境の構築はコストが高く [3]，シナリオの内容や攻撃手法を変更するたびに多大な作業を要する．そのため，既存のサイバーレンジでは少数の固定シナリオに依存する傾向にある [4].

複数の演習を構築する手段として，大規模言語モデル（LLM）にシナリオや Infrastructure as Code（IaC）を自動生成させるアプローチが期待されている．しかし，LLM が生成した演習環境を想定どおりに機能させるには，IaC，実行コード，脆弱性の前提条件などを厳密に整合させる必要があり，事前のプロンプト調整のみでは不十分である．OS やパッケージのバージョン，リソースの起動順序などに起因する不整合は，実行時に初めて明らかになることが多い．特に多段階シナリオにおいては，一つの不整合により後続の手順が実行できなくなるため，実際に検証用の実行コードを動作させ，意図した攻撃が成立することを確認して初めて，環境の妥当性を判断できる．

この課題に対し，先行研究 [1] では，実行ログに基づいて IaC と実行コードを繰り返し修正し，両者の整合性を確保するパイプラインを提案した．具体的には，事前に作成したプロファイルを入力とし，Claude Code CLI を用いて OpenStack の IaC（Heat Orchestration Template: HOT）および実行コードを生成する．その後，これらを OpenStack 環境へデプロイして実行し，得られたログに基づいて修正を繰り返す．本稿では，文献 [1] で STEP 間の受け渡しを人手で実施していた修正のパイプラインを拡張し，一連の制御を Python により自動化する．

本研究の目的は，新規の攻撃手順や脆弱性を発見することではなく，演習環境と実行コードの整合性を保つ構築パイプラインを実現することである．評価では，文献 [1] と同一の Zerologon シナリオを用いて動作を確認する（Z0）．また，初期侵入 3 種と Linux 権限昇格 2 種を組み合わせた計 6 つのシナリオ（S1–S6）を用いて，拡張性を探索的に評価する．各シナリオについて，初回の生成のみで演習が成功した試行数と，実行ログに基づく自動修正を経て成功した試行数を示す．これにより，自動修正により不整合が解消された事例を確認する．

本稿の貢献は次の 3 点である．

- 文献 [1] では人手で行っていた STEP 間の受け渡しと再生成の判断を，Python オーケストレータの固定規則として実装し，生成から展開・実行・修正までを人手の介在なしに反復できるようにした．
- 隔離の成立を LLM の生成物に依存させず，オーケストレータ側の静的検査とネットワークレベルの遮断・到達性検査で機械的に確定する構成を示した．LLM が生成した HOT には実際に 6 件の隔離条件の違反が生じていたが，全 60 試行で演習環境から外部への到達は生じなかった．
- 初期侵入 3 種と権限昇格 2 種を組み合わせた 6 シナリオ 60 試行に適用し，修正ループを有効にした条件下での内訳として，再生成なし成功は 36 試行，最終成功は 59 試行であることを示した．S1–S6 では，HOT 側の不整合はすべて同一 STEP 内の再生成で解消し，STEP3 の診断を経た修正は実行コード側の 7 試行に限られた．

表 1 サイバー演習を構成する目的・シナリオ・演習環境・実行コードの 4 要素．文献 [1] の整理に基づく．提案パイプラインは，このうち演習環境と実行コードの生成・整合を対象とする．

要素	説明
目的	演習提供者が演習受講者に特定の技能や知識を身につけさせることを意図したもの
シナリオ	演習目的に基づいて設計された仮想的な状況と，受講者が取り組む一連の課題
演習環境	シナリオを実現するために用意される仮想マシン，ネットワーク構成，ツール群
実行コード	構築済みの演習環境上でシナリオに定義された手順を遂行・確認するスクリプトやコマンド群．HOT や cloud-init など，演習環境自体を構築するコードは含まない

2. サイバー演習と関連研究

2.1 サイバー演習の構成要素

著者らは文献 [1] で，多段階攻撃を対象とするサイバー演習を，目的，シナリオ，演習環境，実行コードの 4 要素として整理した（表 1）．以降では，この整理に従って演習環境と実行コードを区別して用いる．

2.2 演習環境と IaC

演習環境の構築には IaC が広く用いられており [5], OpenStack では HOT が IaC にあたる．宣言的な記述から演習環境を構築する研究には，CyRIS[6], SecGen[7], KYPO[8] がある．CyRIS は YAML の記述から KVM 上に VM とトポロジを構築する．SecGen はシステムの構成や脆弱性などの構成要素を構造化して指定できる．KYPO はクラウド基盤上に演習環境を構築するが，記述の生成や修正は扱わない．これら 3 件はいずれも，演習環境の構築にとどまり，その上で動作させて攻撃の成立を確認する実行コードの生成は対象外である．著者らも，ブロックプログラミングによる演習環境を構築するシステムを提案している [9]．しかし，これらの既存システムはいずれも，新規シナリオにおける IaC と実行コードの整合性を，実行結果に基づいて動的に修正する機能は備えていない．

2.3 LLM によるシナリオ・環境・攻撃手順生成に関する研究

LLM を用いたサイバー演習に関する研究には，シナリオ記述のみを生成し環境・実行コードを生成しない研究，

© 2026 Information Processing Society of Japan

検査結果に基づいて環境のみを修正する研究，環境を所与として次のコマンドのみを実行結果に基づいて生成する研究，単一 CVE を対象に脆弱 VM またはコンテナと PoC の双方を実行結果に基づいて再現する研究がある．これらの研究をシナリオ生成，演習環境生成，攻撃手順生成，単一 CVE の再現の 4 種類に分ける．表 2 に，各研究が多段階攻撃を対象とするか，環境と実行コードをそれぞれ生成するか，および修正の根拠を示す．

また，LLM による IaC の単発の生成が難しいことは，定量的にも報告されている．Terraform を対象とする IaC-Eval[17] では，GPT-4 でも初回生成の成功率（pass@1）は 19.36%にとどまる．一方で，同研究では実行時のエラーメッセージをフィードバックして自己修正のループを回すことで，成功率が向上する（31.12%）ことも示されている．

本研究は，プロファイルから多段階攻撃の演習環境と実行コードを生成し，実行ログに基づいて両者を修正するパイプラインを提案する．表 2 に示すように，このようなアプローチは調査した範囲では本研究のみである．さらに，複数のシナリオを用いて修正前後の結果を比較し，ループの有効性を検証する点でも関連研究と異なる．

3. 提案パイプライン

3.1 入力と成功条件

パイプラインの入力は，演習を表 3 の 6 種類のプロファイルに分けて記述したものである．ESD の各 step にある technique_ref は，対応する VRP を参照する．EEP の各ノードにある role_tag は，そのノードの構築手順である NCP を参照する．入力形式の具体例は図 A·1–A·4 に示す．

AT は，各ステップの事後条件を順に定義し，最後のステップの事後条件を演習全体の最終目標とする．本稿では，プロファイルで指定した OpenStack のノードとネットワークが正常に作成されたことを確認したうえで，隔離検証を通過し，この最終目標に到達した状態を演習の「成功」と定義する．

3.2 提案パイプラインのフロー

パイプラインを図 1 に示す．LLM と仮想化基盤を仲介し，自動制御するモジュールを，以下オーケストレータと呼ぶ．

LLM の役割は，HOT，実行コード，および修正指示のテキスト生成に限定し，実環境を操作する権限は与えない．一方でオーケストレータが，6 種類のプロファイルからプロンプトを構築し，LLM が生成する．実環境への展開と実行，失敗の検知，再生成経路の選択は，オーケストレータがあらかじめ定めた処理に従って制御する．これにより，生成内容が試行ごとに変わっても，実環境の操作を LLM に任せない安全な運用を実現する．

環境と実行コードのどちらに不整合があるかを特定しやすくするため，本パイプラインを STEP1 から STEP3 の 3 段に分ける．

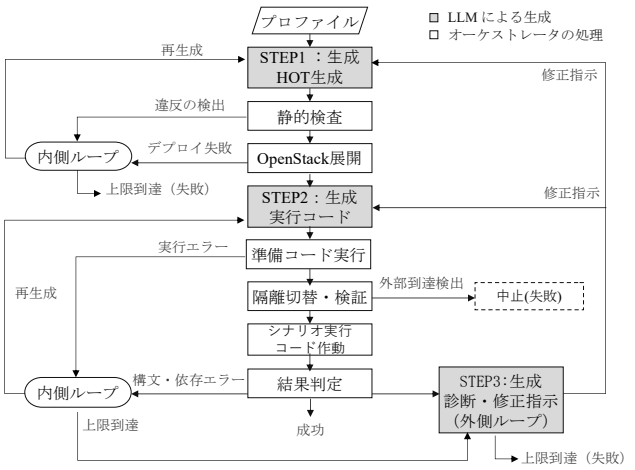


図 1 プロファイルを入力として HOT と実行コードを生成・展開・実行し，結果を根拠に修正するまでの処理．STEP1 は HOT を生成して静的検査を行ったうえで OpenStack 上に展開し，STEP2 はその展開済みの環境と整合する実行コードを生成してその上で実行し，STEP3 はシナリオ実行失敗の原因を診断して修正指示を出す．

STEP1：環境生成　LLM がプロファイルから HOT を生成し，オーケストレータが構文と隔離条件を静的に検査したうえで OpenStack 上に展開する．失敗時は STEP1 内部で再生成（内側ループ）する．

STEP2：実行コード生成　LLM が HOT を参照して展開済みの環境と整合する実行コードを生成する．実行コードは，外部通信の要否に応じて，準備コード（setup.sh）とシナリオ実行コード（scenario_executor.sh）の 2 つに分けて生成され，オーケストレータが前者を隔離前に，後者を隔離後に実行する（隔離手順の詳細は 3.3 節）．シナリオ実行コードは，自らコマンドの終了コードと出力を調べて事後条件を満たしたか確認し，満たさない場合は非ゼロの終了コードで終了するよう設計される．

STEP3：原因分析・修正指示　実行ログと生成物を LLM に入力し，失敗の原因を診断する．LLM の出力は，HOT の再生成指示，実行コードの再生成指示，両方の再生成指示，ゲスト OS 上でコマンドを実行する追加調査指示，手動対応の 5 種類のいずれかである．

失敗を検知した際の修正処理には，内側ループと外側ループの 2 つの経路を設けている．

内側ループ： 単一の生成物に対するエラーを修正する経路である．STEP1 では，OpenStack 上での環境作成失敗をすべて内側ループへ送る．STEP2 では，準備コードの失敗はすべて内側ループへ送る．一方，シナリオ実行コードの失敗は，終了コードや出力の内容から，シェルの構文エラー（終了コード 2），実行権限がない（126），コマンド

© 2026 Information Processing Society of Japan

表 2 LLM を用いたシナリオ・環境・攻撃手順生成に関する関連研究と本研究の比較．「多段階」は，ある攻撃の成功が次の攻撃の前提となるような，依存関係を持つ攻撃チェーンを対象としているかを示す（独立した脅威の並列実行は含まない）．「環境」は，仮想マシンやコンテナ定義，IaC など，デプロイ可能な環境構築アーティファクトを生成するかを示す．「コード」は実行コードを生成するかを示す．「修正の根拠」は，生成物を検証結果に基づき再生成・修正するループにおいて，再生成の要否・内容を判断する根拠を示す．各項目は，論文中に明示的な記述がある場合を ✓，ない場合を一律に — とする．

システム	生成物	多段階	環境	コード	修正の根拠
CyExec[10]（Yamin ら）	演習シナリオ記述	—	—	—	LLM 間の相互推敲
ARCeR[11]	演習環境記述ファイル+デプロイ	—	✓	—	構文検査
AgentCyTE[12]	演習環境記述ファイル+デプロイ	✓	✓	—	スキーマ検証
PentestGPT[13]	次コマンド（環境は所与）	✓	—	✓*	実行結果（次手のみ）
AutoAttacker[14]	次コマンド（環境は所与）	✓	—	✓*	実行結果（次手のみ）
CVE-GENIE[15]	脆弱 VM+PoC	—	✓	✓	実行結果
CVE-Factory[16]	コンテナ+パッチ+検証テスト	—	✓	✓	実行結果
本研究	HOT+実行コード	✓	✓	✓	実行ログ

✓：該当　—：非該当 または 明示的な記述なし　✓*：次に実行する単一コマンドのみの生成

表 3 演習を記述する 6 種類のプロファイル．ESD と EEP はシナリオごとに定め，DEP, NCP, VRP, AT はシナリオ間で共有するライブラリとして存在する．

プロファイル	説明
シナリオ定義（ESD）	演習シナリオを構成する一連の手順として，各ステップの攻撃技術と事後条件を定める
演習環境プロファイル（EEP）	ネットワークトポロジとノード構成を定める
配備環境プロファイル（DEP）	OpenStack 上で利用可能なイメージの仕様として，各ノードに割り当てるイメージとフレーバーを定める
ノード構成プロファイル（NCP）	各ノードに適用する cloud-init 設定など，構築方法に関するナレッジを保持する
脆弱性再現プロファイル（VRP）	脆弱性を CVE 単位で記述し，対象イメージや AT との対応関係など，脆弱性構成のナレッジを保持する
攻撃技術（AT）	攻撃技術の前提条件・事後条件・実行コマンドなど，実行手順に関するナレッジを保持する

が見つからない（127）のように，実行コード自体の欠陥と機械的に判定できる場合にのみ，内側ループへ送る．それ以外は外側ループへ送られる．内側ループでは，直前のエラー出力をプロンプトに付与し，同一 STEP 内で再生成を行う．

外側ループ（STEP3 経由の修正）：STEP2 のシナリオ実行コードの失敗のうち，上記の内側ループの条件に当てはまらない失敗は，すべて STEP3 へ送られる．ログと HOT・実行コード双方を参照し，LLM が失敗原因を診断したうえで，該当する STEP に再生成や追加調査を指示する．

このようなルールを設けることで，単一の生成物が原因のエラーは内側ループで処理され，両方に原因があるかが判断できないエラーのみが STEP3 の外側ループに遷移する．

3.3 隔離設計

シナリオには脆弱性の PoC やデータ破壊が含まれるため，攻撃の実行中は演習ネットワークを外部から完全に隔離する必要がある．LLM の出力には，指示した隔離条件を満たさない記述が含まれる可能性があるため，本システムは生成された HOT の記述のみに依存せず，オーケストレータによる多重の通信遮断と検証を行う．

STEP1 で作成した環境へのパッケージ導入などの準備処理には，外部リポジトリへのアクセスが欠かせない．STEP2 で生成される準備コード（setup.sh）はこの依存関係のインストール・更新を担い，その実行が完了したタイミングでシナリオ実行コードの実行に先立って環境を隔離状態へ移行する．

隔離に向けた準備として，STEP1 ではセキュリティグループを制御する `is_exercise` パラメータを HOT へ組み込むよう LLM に指示する．静的検査では，外部（`0.0.0.0/0`）への通信許可がこのパラメータによる条件分岐の対象となっていることや，VM が外部ネットワークへ直接接続していないことを確認し，違反があれば STEP1 の内側ループへ送る．

検査を通過した HOT を用いた展開後，STEP2 の準備処理が終わると，オーケストレータは `is_exercise` を有効化して外部通信を制限する．さらに，OpenStack Neutron を通じてルータの外部ゲートウェイを直接切断し，ネットワークレベルでの遮断を確実にする．この際，演習対象間の内部通信は維持される．これらの後，環境内の全ルータとポートの設定を調べ，外部経路が一つも存在しないことを確認する．あわせて，全 VM から TCP 接続と名前解決の通信を試行し，外部へ到達できないことを検証する．外部到達を検出した場合は，直ちにパイプラインを中止する．

隔離後の VM は，ネットワークスタックを通らない virtio-serial 経由の QEMU Guest Agent 通信で操作する．

© 2026 Information Processing Society of Japan

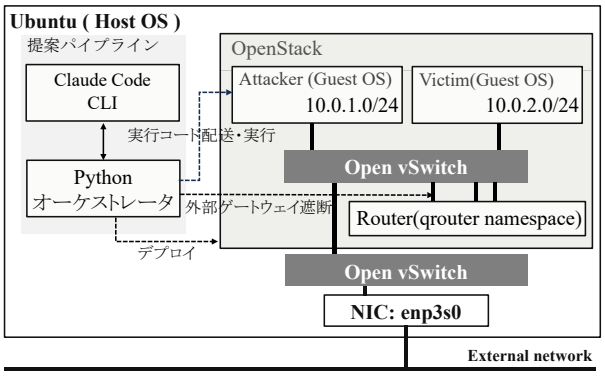


図 2 隔離演習環境の論理構成およびシステム連携のアーキテクチャ．Python オーケストレータがゲートウェイの遮断と実行コードの転送・実行を制御する．

実行コードの送信・実行だけでなく，修正後の再実行や，STEP3 における追加調査コマンドの実行もすべてこの経路で行うため，演習外への意図せぬ通信は発生しない．

なお，VM エスケープや OpenStack の脆弱性を悪用する攻撃は本隔離機構の対象外であり，演習生成の対象にも含めない．

3.4 システムアーキテクチャ

提案パイプラインは，生成を担う LLM の Claude，演習環境の展開先となる仮想化基盤の OpenStack，およびこれらを制御する Python オーケストレータの 3 つから構成される．本システムは Ubuntu 上で動作し，Python オーケストレータが OpenStack と，Claude Code CLI をヘッドレスモードで実行して LLM と連携する．アーキテクチャを図 2 に示す．

環境のデプロイにおいては，オーケストレータが OpenStack のオーケストレーションサービスである Heat API を呼び出し，生成された HOT の定義に従ってネットワーク，攻撃者ノード，標的ノードを展開する．

図 2 の点線矢印が示すように，オーケストレータは展開後の環境に対して，Neutron API を通じたゲートウェイの遮断制御と，QEMU Guest Agent を用いたコード実行を制御する．

4. 評価

4.1 評価方法

本評価では，提案パイプラインの基本機能および自動修正ループの有効性を検証するため，表 4 に示す 7 つのシナリオ（Z0，S1–S6）のプロファイルを入力として用いる．

Z0 は文献 [1] で提案したシナリオであり，これを本研究の自動化パイプラインで再実行することで，パイプラインの基本機能を確認する．S1–S6 は，初期侵入 3 種として公開 Web アプリの Log4Shell（CVE-2021-44228），業務アプリの OFBiz（CVE-2023-51467），リモートアクセスの VPN（設定上の脆弱性），Linux 権限昇格 2 種として PwnKit（CVE-2021-4034），Dirty Pipe（CVE-2022-0847）を組み合わせた 6 シナリオである．パイプラインを複数のシナリオに適用できるか，また，自動修正ループにより失敗を修正できるかを探索的に確認する．

表 4 評価対象のシナリオ一覧．初期侵入と権限昇格を組み合わせ，隔離条件などの他の条件は共通に保つ．

ID	初期侵入	権限昇格
Z0	Zerologon	—
S1	Log4Shell	PwnKit
S2	Log4Shell	Dirty Pipe
S3	OFBiz	PwnKit
S4	OFBiz	Dirty Pipe
S5	VPN	PwnKit
S6	VPN	Dirty Pipe

Z0 は，攻撃者ノード，ドメインコントローラ (DC)，メンバーサーバーの 3 ノードで構成する．S1–S4 では，初期侵入と権限昇格の対象を同一ホストとし，攻撃者ノードと標的ノードの 2 ノードで構成する．S5・S6 では，VPN ゲートウェイを別のホストとして追加し，横展開を経て標的ノードへ到達する．

各シナリオに対して，上限として内側ループ各 3 回，外側ループ 5 回を上限として設定し，それぞれ 10 回試行を繰り返す．

評価には，単一 PC 上に構築した All-In-One 構成の OpenStack 環境を用いた．環境の詳細を付録の表 A·1 に示す．

4.2 評価指標

以下の 9 つの指標を用いる．

再生成なし成功　：シナリオごとの全 10 試行のうち，初回の生成だけで演習に成功した試行数．内側ループおよび STEP3 の診断を経た再生成は発生していない．

最終成功　：全 10 試行のうち，修正の有無によらず最終的に演習に成功した試行数．再生成なし成功は最終成功の部分集合である．

内側 HOT 修正試行数　：STEP1 の内側ループにおける再生成が 1 回以上発生した試行数（10 試行中）．

内側実行コード修正試行数　：STEP2 の内側ループにおける再生成が 1 回以上発生した試行数（10 試行中）．

外側 HOT 修正試行数　：STEP3 の診断を経由した HOT 再生成が 1 回以上発生した試行数（10 試行中）．

外側実行コード修正試行数　：STEP3 の診断を経由した実行コード再生成が 1 回以上発生した試行数（10 試行中）．

所要時間（分）　：試行あたりの所要時間の平均値．LLM のトークン制限待ちなどの待機時間を除外している．

トークン数　：試行あたりの使用トークン数の平均値．

料金（USD）　：試行あたりの API 利用料金の平均値．

© 2026 Information Processing Society of Japan

Claude から返却される値であり，モデルの価格改定などに依存する．

4.3 Z0 の評価結果

シナリオごとの評価結果を表 5 に示す．Z0 について，10/10 で最終目標を達成した．このうち，再生成なし成功は 1 試行であり，残り 9 試行は自動修正を経て成功した．なお，文献 [1] で確認された入力プロファイル記述の不備は，本評価で用いたプロファイルではすでに修正済みである．

一方，再生成なし成功が 1 試行にとどまった要因として，本実装にあわせて STEP1 の HOT 生成プロンプトへ新たに追加した，複数ネットワーク間の直接到達を原則禁止する隔離要件が挙げられる．この要件により，攻撃者と被害者間のセキュリティグループが初回生成では自己参照ルールのみとなり，Zerologon が前提とする攻撃者ノードから DC への直接到達（TCP 135/445）が遮断される事例が 9 試行で発生した．これらはいずれも STEP3 の診断により通信許可ルールの追加を指示され，最終的に全 10 試行が成功に至っている．これらは本実装で追加した隔離要件が原因であり，シナリオの問題ではない．

4.4 S1–S6 の評価結果

S1–S6 を通じて，再生成なし成功は 36/60（60%），最終成功は 59/60（98%）であり，自動修正により 23 件が成功に至った．最も成功数が増加したのは S1（Log4Shell × PwnKit）で，再生成なし成功 4/10 から最終成功 10/10 まで向上した．一方，S4（OFBiz × Dirty Pipe）は再生成なし成功の時点で 8/10 と高く，改善は 2 件にとどまった．失敗した 1 件は S3（OFBiz × PwnKit）であり，シナリオの欠陥ではなく，コード生成中の `claude -p` プロセスの異常終了（API 通信エラー）によるものであった．

修正が発生した試行数を内側ループと外側ループに分けると，HOT 側は内側 16 試行，外側 0 試行であり，修正の多くは内側ループで行われた．一方，実行コード側は内側 2 試行，外側 7 試行に分かれた．この違いは，3.2 節で定めた振り分け規則によるものであり，外側で実行コードを修正した 7 試行は，環境側と実行コード側のどちらにも原因があり得ると判断された失敗である．

外側実行コード修正が生じた S1，S3，S4 のうち，該当した計 7 試行はいずれも最終成功に至った．なお，唯一失敗に終わった S3 の 1 件は，再生成回数は全てゼロであり，この外側実行コード修正の対象には含まれていない．

5. 考察

5.1 実行ログに基づく不整合の解消

以下に，修正ループを経て成功に至った 23 件のうち，STEP3 の指示により失敗を修正した事例を示す．

S3（OFBiz × PwnKit）の試行では，生成された実行コードの構文は正しいものの，攻撃ロジックと事後条件の検証ロジックの間に不整合が生じる事例が複数確認された．例えば，ある試行では，初期侵入の実行結果を持った HTTP レスポンスを `curl -o` を用いてファイルへ保存する処理が生成された．しかし，成否判定の処理は，常に空となるシェル変数を参照するよう記述されていたため，攻撃自体は成功しているにもかかわらず，判定は失敗となった．STEP3 の診断プロセスは，実際の実行ログと出力ファイルの状態からこの不整合を特定し，判定対象を変数から出力ファイルへ切り替える修正を指示して成功に導いた．

また別の S3 の試行では，SSH 公開鍵を `authorized_keys` へ登録するステップにおいて，検証ロジックが誤った基準を用いていたため失敗と判定された．この際，STEP3 は対象ノードへ直接 `ls` や `cat` コマンドを実行して鍵が正しく登録されている実態を確認し，検証ロジック側の誤りであると見抜いて修正した．これらの事例は，生成コード自身の誤りに対し，実際の実行結果と修正方法を指示する本パイプラインの手法が有効に機能したことを示している．

S4 の別の試行では，対象ノード (OFBiz サーバー) の初回起動処理が Heat の規定する待機時間を超過し，スタック構築が一旦失敗する事象が発生した．オーケストレータは QEMU Guest Agent を介してログを確認し，タイムアウト発生後も OFBiz が構築処理を継続中であり，プロセス自体は異常終了していないことを突き止めた．この診断結果に基づき，STEP3 は HOT の待機時間を，実際に OFBiz の起動にかかった時間より長めの値へ変更するよう指示し，再デプロイにより構築は成功した．

この事例における問題の本質は，コード記述の誤りではなく，OFBiz の初回起動処理に要する実時間が試行ごとに変動し，テキスト生成の時点では正確に見積もれない点にある．この修正は，静的なコードレビューだけでは特定できず，実際に実行して初めて可能になったといえる．

5.2 失敗要因の分析と自動修正の限界

最終的に成功に至らなかった 1 試行は，Claude Code 側で発生したエラーにより `claude -p` プロセスが異常終了したことが原因であった．

また LLM によるテキスト生成の能力向上だけでは解決できない，実行時の診断情報が失われることに起因する限界も確認された．S3 の別の試行において，OpenStack の Heat による対象 VM のデプロイが，構築処理の途中で異常終了する事象が発生した．この際，構築失敗の詳細を Heat へ報告する仕組みが機能せず，エラーの詳細情報が欠落した状態となった．これに対し，原因究明のため，QEMU Guest Agent を経由して診断コマンドを実行しようと試みた．しかし，この時点で QEMU Guest Agent も応答せず，内部情報を取得できなかった．

© 2026 Information Processing Society of Japan

表 5 Z0，S1–S6 ごとの試行結果．再生成なし成功は初回の生成だけで成功した試行数，最終成功は修正後を含む成功試行数である．修正の各列は，その種別の再生成が 1 回以上発生した試行数（10 試行中）であり，1 試行内で複数回発生した場合も 1 として数える．内側ループは STEP1・STEP2 内，外側ループは STEP3 の診断後に行った再生成を表す．所要時間，トークン数，料金は試行あたりの平均値である．

		成功数		内側ループ修正試行数		外側ループ修正試行数		試行あたり平均		
ID	試行数	再生成なし	最終	HOT	実行コード	HOT	実行コード	時間（分）	トークン数	料金（USD）
Z0	10	1	10	5	0	9	1	49.6	69660	3.743
S1	10	4	10	4	0	0	2	19.1	25665	1.290
S2	10	7	10	2	1	0	0	17.9	26571	1.118
S3	10	4	9	1	0	0	4	35.8	30035	1.409
S4	10	8	10	2	0	0	1	38.1	40671	1.523
S5	10	6	10	4	0	0	0	16.2	32148	1.210
S6	10	7	10	3	1	0	0	15.8	33409	1.220

この試行は，情報が欠落したまま LLM が推測に基づいて HOT の記述を書き直し，3 回目のデプロイで成功した．しかし，これは実行ログに基づく修正の結果とは言えない．

5.3 演習環境における隔離の成立

デプロイ前の HOT 静的検査では，セキュリティグループの全開放を含む 6 件の隔離条件の違反を検出し，STEP1 の内側ループで修正した．その後，オーケストレータが外部ゲートウェイを切断し，外部経路と到達性を検査した．全 60 試行で外部への到達は確認されず，隔離検証によるシナリオの中止も 0 件であった．これらの結果は，LLM の生成能力のみに依存せず，システム側の静的・動的検査を組み合わせることで，隔離条件の違反を検出して修正し，演習環境の隔離を維持できることを示している．

5.4 修正ループのコスト分析

全 60 試行の総費用（$77.70）のうち，自動修正に費やされたコストは全体の約 26%（$20.34）であった．修正が発生したのは 23 試行であり，1 試行あたりの追加費用は約 $0.88 となる．人手の介在なしにこれらの試行を自動実施できたことで，提案パイプラインの STEP 間制御の自動化による効果を確認した．

5.5 本研究の限界

本評価の全試行は，修正ループを有効にした条件下で実施している．再生成なし成功は，このうち修正が一度も発生しなかった試行の内訳であり，修正を無効にした条件を独立に実施して比較した結果ではない．

本評価は単一の OpenStack 環境と単一の LLM モデルを用いた，各シナリオ 10 試行の探索的評価であり，他の環境・モデルへの一般化可能性の検証は今後の課題である．また，各ステップの事後条件を確認する判定処理は，STEP2 で LLM が生成する実行コードの内部にある．判定処理そのものが誤って生成された場合，事後条件を満たしていない試行を成功として記録する可能性があるが，本稿ではこの可能性を検証していない．

入力となる 6 種のプロファイルは別途作成する必要がある．環境構築の一部を自動化しても，必要な知識をプロファイルとして構造化する作業は残る．ただし，作成したプロファイルは再利用でき，本評価の S1–S6 のように，既存のプロファイルを組み合わせて新たなシナリオを構築できる．しかし，シナリオを拡張する際には，OS やパッケージのバージョンの衝突，特定のサービスを同じ環境で実行できないことなどの理由によりプロファイル間の競合が生じる可能性がある．ただし，今回の S1–S6 では，この競合は確認されなかった．本パイプラインはこうしたプロファイル間の競合を環境構築前に検出する機能を持たないため，プロファイルの組み合わせは手動で確認する必要がある．

6. おわりに

サイバー演習の構築では環境と実行コードの整合を保つ必要があるが，多段階の演習を対象に両者を生成し，実行結果に基づいて整合を修正する研究は，調査した範囲では確認できなかった．

本稿では，プロファイルを入力として HOT と実行コードを生成・展開・実行し，その実行ログを根拠として生成物を自動修正するパイプラインを実装した．文献 [1] の STEP 間の受け渡しを Python オーケストレータにより自動化し，人手の介在なしに 60 試行の評価を自動実施できた．

隔離については，LLM の出力に依存せず，デプロイ前の静的検査とネットワークレベルの遮断・到達性検査によって成立を確定する機構を組み込んだ．静的検査は 6 件の隔離条件の違反を検出して内側ループで是正し，全 60 試行を通じて外部への到達は生じなかった．

評価として，単一の Zerologon シナリオ（Z0）を用いた動作確認において，パイプラインの基本機能が意図通り動作することを確認した．さらに，初期侵入 3 種と Linux 権限昇格 2 種を組み合わせた 6 シナリオ（S1–S6）計 60 試行の評価では，初回の再生成なし成功は 36 試行であり，実行ログに基づく自動修正ループを経た最終成功は 59 試行であった．ただしこれは，単一の OpenStack 環境と単一の

© 2026 Information Processing Society of Japan

LLM モデルを用いた各シナリオ 10 試行の探索的評価であり，修正を無効にした条件との独立した比較ではない．

最終成功に至らなかった 1 件は，生成物の欠陥ではなく LLM 実行基盤側の異常終了によるものであった．試行あたりの平均所要時間は 15.8 分から 38.1 分，平均料金は $1.118 から$1.523 であった．

今後は，シナリオの拡張やプロファイルの追加により，多様な多段階攻撃を扱う演習を対象として評価する．あわせて，事後条件の判定処理そのものの妥当性を検証したうえで，修正を無効にした条件を独立に実施し，修正の効果と限界を検証する．

謝辞　謝辞本研究は JST 経済安全保障重要技術育成プログラム JPMJKP24K1 および JSPS 科研費 JP23K03847 の成果を含みます．

参考文献

[1] 城 明日香，毛利 公美，瀧田 愼，葛野 弘樹，白石 善明：多段階攻撃シナリオに基づくサイバー演習環境と実行コードの自動生成，情報処理学会マルチメディア，分散，協調とモバイル（DICOMO2026）シンポジウム論文集，Vol.2026，pp.285–293（2026）．

[2] Konak, A.: Experiential Learning Builds Cybersecurity Self-Efficacy in K-12 Students, Journal of Cybersecurity Education, Research and Practice, Vol.2018, No.1, Article 6 (2018).

[3] Orbinato, V.: A Next-generation Platform for Cyber Range-as-a-Service, Proc. 2021 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW), pp.314–318 (2021).

[4] Costa, G., Russo, E. and Armando, A.: Automating the Generation of Cyber Range Virtual Scenarios with VSDL, Journal of Wireless Mobile Networks, Ubiquitous Computing, and Dependable Applications, Vol.13, No.4, pp.61–80 (online), DOI: 10.58346/JOWUA.2022.I4.004 (2022).

[5] Chouliaras, N., Kittes, G., Kantzavelou, I., Maglaras, L., Pantziou, G. and Ferrag, M.A.: Cyber Ranges and TestBeds for Education, Training, and Research, Applied Sciences, Vol.11, No.4, Article 1809 (online), DOI: 10.3390/app11041809 (2021).

[6] Beuran, R., Pham, C., Tang, D., Chinen, K., Tan, Y. and Shinoda, Y.: Cybersecurity Education and Training Support System: CyRIS, IEICE Transactions on Information and Systems, Vol.E101-D, No.3, pp.740–749 (2018).

[7] Schreuders, Z.C., Shaw, T., Shan-A-Khuda, M., Ravichandran, G., Keighley, J. and Ordean, M.: Security Scenario Generator (SecGen): A Framework for Generating Randomly Vulnerable Rich-scenario VMs for Learning Computer Security and Hosting CTF Events, Proc. 2017 USENIX Workshop on Advances in Security Education (ASE 17), USENIX Association (2017).

[8] Vykopal, J., Ošlejšek, R., Čeleda, P., Vizváry, M. and Tovarňák, D.: KYPO Cyber Range: Design and Use Cases, Proc. 12th International Conference on Software Technologies (ICSOFT 2017), Vol.1, pp.310–321 (2017).

[9] 城 明日香，毛利 公美，白石 善明：ハンズオンサイバー演習のためのブロックプログラミングを用いた演習環境構築システム，情報処理学会マルチメディア，分散，協調とモバイル（DICOMO2025）シンポジウム 2025 論文集，Vol.2025，pp.1095–1104（2025）．

[10] Yamin, M.M., Hashmi, E., Ullah, M. and Katt, B.: Applications of LLMs for Generating Cyber Security Exercise Scenarios, IEEE Access, Vol.12, pp.143806–143822 (2024).

[11] Lupinacci, M., Blefari, F., Romeo, F., Pironti, F.A. and Furfaro, A.: ARCeR: An Agentic RAG for the Automated Definition of Cyber Ranges, Proc. Availability, Reliability and Security: ARES 2025 International Workshops, pp.23–40, Springer (2025).

[12] Rodriguez, A.M., Acosta, J., Kotal, A. and Piplai, A.: AgentCyTE: Leveraging Agentic AI to Generate Cybersecurity Training & Experimentation Scenarios, Proc. 2025 Annual Computer Security Applications Conference Workshops (ACSAC Workshops), pp.451–458 (2025).

[13] Deng, G., Liu, Y., Mayoral-Vilches, V., Liu, P., Li, Y., Xu, Y., Zhang, T., Liu, Y., Pinzger, M. and Rass, S.: PentestGPT: Evaluating and Harnessing Large Language Models for Automated Penetration Testing, Proc. 33rd USENIX Security Symposium, pp.847–864, USENIX Association (2024).

[14] Xu, J., Stokes, J.W., McDonald, G., Bai, X., Marshall, D., Wang, S., Swaminathan, A. and Li, Z.: AutoAttacker: A Large Language Model Guided System to Implement Automatic Cyber-attacks, arXiv:2403.01038 (2024).

[15] Ullah, S., Balasubramanian, P., Guo, W., Burnett, A., Pearce, H., Kruegel, C., Vigna, G. and Stringhini, G.: From CVE Entries to Verifiable Exploits: An Automated Multi-Agent Framework for Reproducing CVEs, arXiv:2509.01835 (2025).

[16] Luo, X., Zhang, J., Zhou, S., Huang, J., Xiao, C., Zhu, Q., Ma, Z., Yue, X., Yue, Y., Zeng, W. and Che, W.: CVE-Factory: Scaling Expert-Level Agentic Tasks for Code Security Vulnerability, arXiv:2602.03012 (2026).

[17] Kon, P.T.J., Liu, J., Qiu, Y., Fan, W., He, T., Lin, L., Zhang, H., Park, O.M., Elengikal, G.S., Kang, Y., Chen, A., Chowdhury, M., Lee, M. and Wang, X.: IaC-Eval: A Code Generation Benchmark for Cloud Infrastructure-as-Code Programs, Proc. 38th Conference on Neural Information Processing Systems (NeurIPS 2024) Datasets and Benchmarks Track (2024).

研究倫理

本稿のパイプラインは，専用の OpenStack テナント上に構築した隔離環境でのみ実施した．演習ネットワークから演習外への通信は実行前に遮断し，実運用システムには到達させない．

評価に用いるシナリオは，既に公開されている脆弱性および関連する公開情報を，システムの適用範囲を示すテストワークロードとして扱うものである．新規の脆弱性の発見や，未公開の攻撃手法の提示を目的としない．製品名・サービス名・CVE 識別子を記載する場合は，既知の公開情報として扱う範囲に限り，本稿の評価条件を特定するために必要な最小限にとどめる．

本稿および付録には，実行可能な攻撃コード，実環境へ転用可能な詳細な攻撃手順，実 IP アドレス，実パスワー

© 2026 Information Processing Society of Japan

ドを記載しない．公開されている PoC を参照する場合は，NCP に入手元と版を固定して記述し，内容を確認したうえで，隔離環境内でのみ実行する．成功判定は，隔離環境内で完結し実環境に影響を及ぼさない最終目標に基づき定義する．

本システムを公開した場合，隔離の前提を外し，攻撃環境の自動生成へ転用されるおそれがある．このリスクを抑えるため，閉域での実行，実データの不使用，および公開物からの詳細な攻撃手順の除外を前提とする．

付　　録

A.1　実行環境の構成

4 章の評価に用いた実行環境の構成を表 A·1 に示す．評価は，この単一の All-In-One 構成の OpenStack 環境上で，Z0 と S1–S6 の全 70 試行を通して同一の構成のまま実施した．

表 A·1　評価に使用した実行環境の構成．

項目	仕様
CPU	Intel® Core™ i9-12900HK
メモリ	32GB
ストレージ	SSD 1TB
OS	Ubuntu 24.04.2 LTS
OpenStack シリーズ	Caracal

A.2　プロファイルの抜粋

本付録では，3.1 節および表 3 で述べた 6 種類のプロファイルのうち，ESD，EEP，VRP，AT の YAML の抜粋を一部編集し，図 A·1–A·4 に入力形式の具体例として示す．AT に含まれる各 step の実行コマンド（command）および攻撃手順の詳細は，セキュリティ上の配慮から省略し，該当箇所を「(省略)」と表記する．図 A·4 は，図 A·1 が示す step 1 ではなく，Zerologon 攻撃の後続 step で参照される AT の例である．

```
step:  1
tactic:  "Initial Access"
technique:  "T1190 - Exploit Public-Facing Application"
cve:  "CVE-2020-1472"
technique_ref:  "vuln_profiles/CVE-2020-1472.yaml"
execution_node:  "attacker"
source_node:  "attacker"
target_node:  "dc"
attacker_privilege:
    execution_node:  "root"
    target_node:  "null"
pre_condition:  "攻撃者が DC の Netlogon サービス（TCP 135/445）にネットワーク的に到達可能"
post_condition:  "DC のマシンアカウントパスワードが空に変更完了"
target_service:  "Samba 4.10.17"
```

図 A·1　シナリオ定義（ESD）の抜粋．scenarios/samba-zerologon/Exercise_Scenario_Definition.yaml の step 1.

```
networks:
    - name:  "attacker-net"
      cidr:  "10.0.1.0/24"
    - name:  "victim-net"
      cidr:  "10.0.2.0/24"
nodes:
    - id:  "attacker"
      role:  "攻撃者マシン"
      network:  "attacker-net"
      role_tags:  []
      required_by:  [1, 2, 3, 4, 5]
    - id:  "dc"
      role:  "Samba AD Domain Controller"
      network:  "victim-net"
      role_tags:  ["samba_ad_dc"]
      required_by:  [1, 2, 3, 4]
    - id:  "member_server"
      role:  "ドメイン参加済みメンバーサーバー"
      network:  "victim-net"
      role_tags:  ["sssd_kerberos_ssh"]
      required_by:  [4, 5]
```

図 A·2　演習環境プロファイル（EEP）の抜粋．scenarios/samba-zerologon/Exercise_Environment_Profile.yaml の topology 以下.

© 2026 Information Processing Society of Japan

```
cve:  CVE-2020-1472
name:  Zerologon
description:  "Netlogon プロトコルの暗号化の欠陥により，DC のマシンアカウントパスワードを空に変更可能"
pre_conditions:
    - "攻撃者が DC の Netlogon サービス（TCP 135/445）にネットワーク的に到達可能"
    - "ターゲットが Samba AD DC として稼働"
    - "Samba バージョンが 4.10.18 未満"
post_condition:
    - "DC のマシンアカウントパスワードのリセット完了"
version_pin:
    samba:  "4.10.17"
image_ref:  "ubuntu-20.04-samba-ad-dc"
```

図 A·3 脆弱性再現プロファイル（VRP）の抜粋．vuln_profiles/CVE-2020-1472.yaml．

```
name:  "vampire keytab による横展開"
pre_conditions:
    - "DC のマシンアカウントのパスワードが判明している"
    - "攻撃者が DC の SMB サービス（TCP 445）にネットワーク的に到達可能"
post_condition:
    - "Administrator のハッシュ値を取得"
    - "krbtgt のハッシュ値を取得"
attack_chain:
    - step:  1, name:  "（省略）"
    - step:  2, name:  "（省略）", technique:  "T1003"
    - step:  5, name:  "（省略）", technique:  "T1550.002"
```

図 A·4 攻撃技術（AT）の抜粋．attack_techniques/vampire_keytab_lateral.yaml．

© 2026 Information Processing Society of Japan